DHBW Heilbronn

Software Engineering II.1 – Block 7

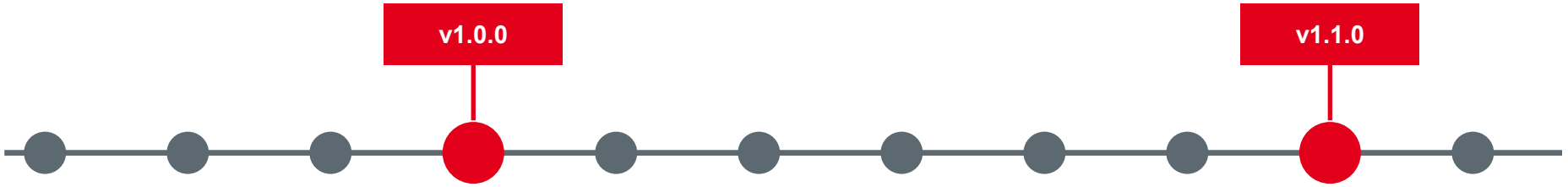
Management und Abschlusspräsentationen

Release verstehen, Risiken bewerten, Entscheidung begründen – danach Ihre Präsentationen

Software Engineering II.1 | WS 2026/27 | Dr. rer. nat. Ana Nicolaescu

Was ist ein Release?

Ein Release ist ein fester, getesteter Stand des Projekts, den die Nutzerinnen bekommen.



Jeder Punkt ist ein Commit auf main: täglich neue Stände, manche halbfertig.

Das Etikett heißt „Tag“

Ein Tag klebt an genau einem Commit und ändert keinen Code. Der Commit bleibt für immer derselbe, auch wenn main weiterläuft.

Wozu?

Später genau diesen Stand wieder bauen. Eine Nutzerin mit 1.0.0 meldet einen Fehler: Sie holen v1.0.0 und stellen ihn nach, nicht den heutigen Stand.

Tag = Etikett auf einem Commit. Release = ein getesteter Stand, der so ein Etikett trägt.

Die Versionsnummer lesen: MAJOR . MINOR . PATCH

1
MAJOR

WANN ERHÖHEN?

Etwas Altes passt nicht mehr. Nutzerinnen müssen etwas tun.

BEISPIEL PHOTO ALBUM

Neues Dateiformat: alte Alben lassen sich nicht mehr öffnen.

ab 1.0.0: → 2.0.0

1
MINOR

WANN ERHÖHEN?

Neue Funktion. Alles Alte funktioniert weiter.

BEISPIEL PHOTO ALBUM

Tags für Fotos (Change Request CR-1).

ab 1.0.0: → 1.1.0

0
PATCH

WANN ERHÖHEN?

Nur ein Fehler behoben. Nichts Neues.

BEISPIEL PHOTO ALBUM

Duplikaterkennung repariert.

ab 1.0.0: → 1.0.1

Es zählt nur eine Zahl hoch. Die Zahlen rechts davon werden auf 0 gesetzt: 1.0.1 → 1.1.0 → 2.0.0.

Übung: Welche Versionsnummer?

Übung

Das aktuelle Release ist 1.0.0. Welche Nummer bekommt das nächste?

a	Die Duplikaterkennung hatte einen Fehler. Er ist behoben, sonst hat sich nichts geändert.	**1.0.1**	**1.1.0**	**2.0.0**
b	Neu: Tags für Fotos. Alte Alben funktionieren unverändert weiter.	**1.0.1**	**1.1.0**	**2.0.0**
c	Neues Dateiformat. Alte Alben lassen sich nicht mehr öffnen, die Nutzerin muss sie selbst konvertieren.	**1.0.1**	**1.1.0**	**2.0.0**

Erst allein entscheiden, dann gemeinsam zeigen: erste, zweite oder dritte Nummer.

Übung: Welche Versionsnummer? · Lösung

Übung

Das aktuelle Release ist 1.0.0. Welche Nummer bekommt das nächste?

a	Die Duplikaterkennung hatte einen Fehler. Er ist behoben, sonst hat sich nichts geändert.	**1.0.1**	1.1.0	2.0.0
b	Neu: Tags für Fotos. Alte Alben funktionieren unverändert weiter.	1.0.1	1.1.0	2.0.0
c	Neues Dateiformat. Alte Alben lassen sich nicht mehr öffnen, die Nutzerin muss sie selbst konvertieren.	1.0.1	1.1.0	2.0.0

Gleich im Projekt: Neue Funktion plus automatische Umstellung der Alben = 1.1.0. Müsste die Nutzerin selbst etwas tun: 2.0.0.

Übung: Welche Versionsnummer? · Lösung

Übung

Das aktuelle Release ist 1.0.0. Welche Nummer bekommt das nächste?

a	Die Duplikaterkennung hatte einen Fehler. Er ist behoben, sonst hat sich nichts geändert.	1.0.1	1.1.0	2.0.0
b	Neu: Tags für Fotos. Alte Alben funktionieren unverändert weiter.	1.0.1	**1.1.0**	2.0.0
c	Neues Dateiformat. Alte Alben lassen sich nicht mehr öffnen, die Nutzerin muss sie selbst konvertieren.	1.0.1	1.1.0	2.0.0

Gleich im Projekt: Neue Funktion plus automatische Umstellung der Alben = 1.1.0. Müsste die Nutzerin selbst etwas tun: 2.0.0.

Übung: Welche Versionsnummer? · Lösung

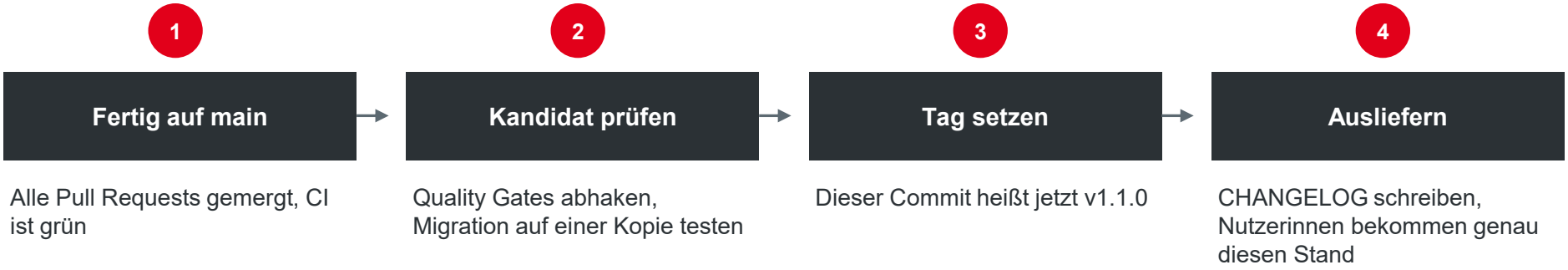
Übung

Das aktuelle Release ist 1.0.0. Welche Nummer bekommt das nächste?

a	Die Duplikaterkennung hatte einen Fehler. Er ist behoben, sonst hat sich nichts geändert.	1.0.1	1.1.0	2.0.0
b	Neu: Tags für Fotos. Alte Alben funktionieren unverändert weiter.	1.0.1	1.1.0	2.0.0
c	Neues Dateiformat. Alte Alben lassen sich nicht mehr öffnen, die Nutzerin muss sie selbst konvertieren.	1.0.1	1.1.0	**2.0.0**

Gleich im Projekt: Neue Funktion plus automatische Umstellung der Alben = 1.1.0. Müsste die Nutzerin selbst etwas tun: 2.0.0.

Vom Commit zum Release in vier Schritten



SCHRITT 3 IM TERMINAL

```
git tag -a v1.1.0 -m "Release 1.1.0"
git push origin v1.1.0
```

Zeile 1 legt das Etikett auf Ihrem Rechner an.

Zeile 2 lädt es ins Remote hoch. Ohne diese Zeile sieht das Team das Etikett nicht.

Ein Fehler später? Neuer Commit, neues Tag v1.1.1. Das alte Tag bleibt.

Das Tag ist kein Branch und ändert keinen Code. Es benennt einen Commit, damit man ihn später wiederfindet.

Release ist nicht Rollout: vier Wege zu den Nutzerinnen

Release = Stand festlegen (Tag). **Rollout** = diesen Stand an die Nutzerinnen verteilen.

Big Bang



Anteil neue Version, Tag 1

Alle 5.000 am Tag 1

PASST BEI

kleine, gut getestete Änderung

RISIKO

Fehler trifft alle, kein Zurück

Pilotbetrieb



Anteil neue Version, Tag 1

Erst 100, dann alle

PASST BEI

Datenmigration wie bei 1.1.0

RISIKO

Pilotgruppe nicht repräsentativ

Stufenweise



Anteil neue Version, Tag 1

Wellen: 20 %, 50 %, 100 %

PASST BEI

große Nutzerbasis

RISIKO

lange Übergangszeit, alt und neu müssen zusammenpassen

Parallelbetrieb



Anteil neue Version, Tag 1

Alt und neu laufen nebeneinander

PASST BEI

kritische Systeme

RISIKO

doppelte Kosten und Pflege

Faustregel: Je größer der mögliche Schaden und je schwieriger der Rückweg, desto vorsichtiger ausrollen.

Übung: Welcher Rollout für 1.1.0?

Handzeichen

Photo Album 1.1.0 geht an 5.000 bestehende Nutzerinnen. Alle ihre Alben müssen auf das neue Modell umgestellt werden (Foto in mehreren Alben).

2 min

Plenum

A Big Bang	B Pilotbetrieb	C Stufenweise	D Parallelbetrieb

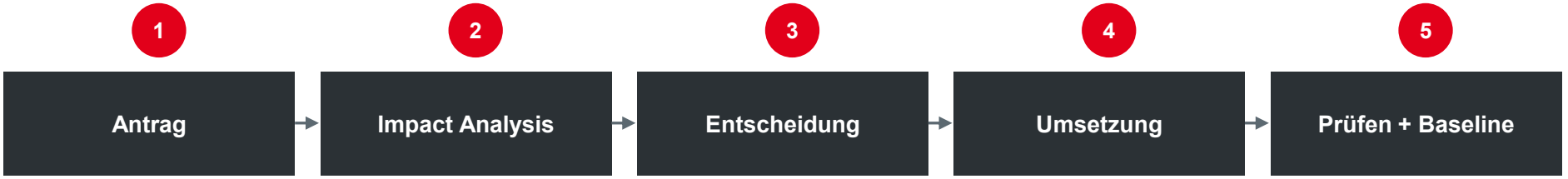
AUFTRAG

1. Entscheiden Sie allein und zeigen Sie auf Zuruf A, B, C oder D.
2. Im Team (1 Minute): Was ist das Schlimmste, das passieren kann, und was tun Sie dagegen?

ERGEBNIS Eine Wahl und ein Worst-Case-Szenario mit Gegenmaßnahme.

DANACH IM PLENUM Ein Team nennt sein Szenario, wir sammeln die Antworten.

Änderungen kontrolliert: fünf Schritte



Baseline (Grundlinie) = ein benannter, geprüfter Stand

Das ist genau das, was ein Tag festhält. Ihre Tags block-2 bis block-6 und v1.0.0 sind Baselines. Eine Baseline ändert man nur über diesen Prozess, nicht „mal schnell“.

Erst analysieren, dann entscheiden, dann umsetzen – so wie Sie es mit CR-1 gemacht haben.

Quality Gates: Ja/Nein-Fragen mit Beleg

Kein Gate: „Der Code ist gut.“

Gate: „Branch Coverage der Domäne ≥ 90 %, Report verlinkt.“

☐	**PR → main**	CI grün und mindestens ein Review
☐	**Alle muss-Anforderungen**	haben mindestens einen Test (Traceability)
☐	**Branch Coverage Domäne**	≥ 90 %, JaCoCo-Report verlinkt
☐	**Kein offener Bug**	mit Priorität „hoch“
☐	**Migration**	auf einer Kopie echter Daten getestet, Rollback geprobt

Projektbeschreibung §13: Wählen Sie 4 bis 6 Gates, die Sie wirklich prüfen können, und belegen Sie sie mit einem Link.

Risikomanagement in drei Schritten

1 Erkennen

Was könnte schiefgehen?

- Systematisch suchen, nicht aus dem Bauch
- Anforderungen, Architektur, Tests, Team (Bus-Faktor), Technik, Betrieb (Migration), KI

2 Bewerten

Wie wahrscheinlich, wie schlimm?

- Wahrscheinlichkeit × Auswirkung, je 1 bis 3, Ergebnis 1 bis 9
- Ehrlich sein: „hoch/hoch“ ist erlaubt

3 Minimieren

Was tun wir?

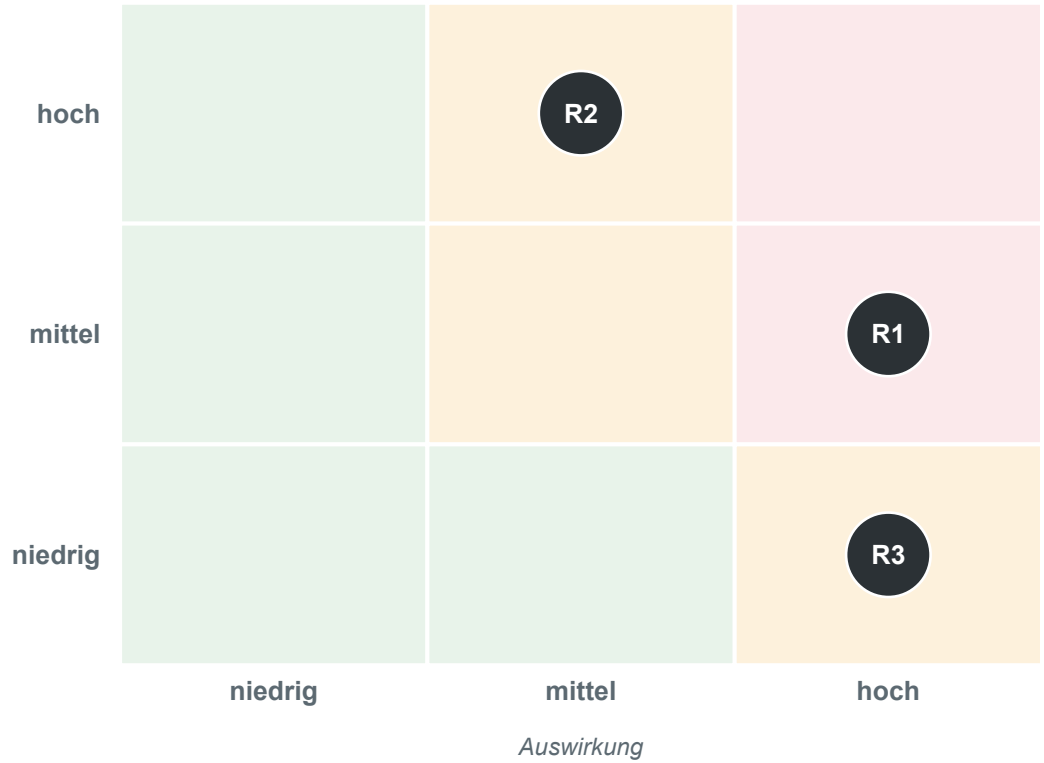
- Vermeiden: Feature streichen
- Vermindern: Test, Backup, Review
- Übertragen: Bibliothek, Dienst
- Akzeptieren: dokumentiert, mit Trigger

Produktrisiko Fehler, Sicherheit oder Datenverlust bei den Nutzerinnen

Projektrisiko Termin, Team oder Aufwand des Projekts

Risiko = Eintrittswahrscheinlichkeit × Auswirkung. Je Risiko eine bewusste Strategie.

Risikomatrix: Beispiel Photo Album 1.1.0



R1 Migration 1.0 → 1.1 verliert Fotozuordnungen

W mittel · A hoch = 6 · Produkt

Vermindern: Migrationstest mit Kopie echter Daten, Backup, Rollback-Skript, Pilot

R2 Duplikatregel je Album bei geteilten Fotos falsch

W hoch · A mittel = 6 · Produkt

Vermindern: Entscheidungstabelle erweitern, Tests je Album, Review

R3 Nur eine Person kennt FileAlbumRepository

W niedrig · A hoch = 3 · Projekt

Vermindern: Pair-Review, ADR + Sequenzdiagramm aktuell; akzeptieren mit Trigger „Person fällt aus“

Release-Entscheidung = Matrix + Quality Gates + ehrliche Antwort: „Ja, unter der Bedingung …“ oder „Nein, bis …“

Risiko-Check: Würden Sie releasen?

Teamarbeit

Projektbeschreibung §13: Würden Sie die aktuelle Version freigeben? Beantworten Sie das mit Belegen.

9 min

AUFTRAG

1. Erkennen (3 min): Nennen Sie drei Risiken, mindestens ein Produkt- und ein Projektrisiko. Ideen: Coverage-Lücken, offene Issues, Migration, Bus-Faktor, KI-Funktion.
2. Bewerten und minimieren (3 min): Wahrscheinlichkeit × Auswirkung mit einem Satz Begründung. Je Risiko eine Strategie und eine Maßnahme.
3. Entscheiden (3 min): Ja, Nein oder Ja unter Bedingung. Nennen Sie die Matrix und mindestens zwei Quality Gates, jeweils mit Link.

ERGEBNIS Entwurf in RISK_ANALYSIS.md. Den Rest schreiben Sie bis zur finalen Abgabe.

DANACH IM PLENUM Zwei Teams nennen ihre Entscheidung: Ja, Nein oder unter Bedingung.

Crawl – Walk – Run: KI-Autonomie schrittweise erhöhen

Crawl: vorschlagen

KI schlägt vor, der Mensch entscheidet.

Beispiel: Die KI schlägt eine Fotobeschreibung vor, die Nutzerin bestätigt.

Messwert: Übernahmerate

Walk: mit Freigabe handeln

KI handelt, Unumkehrbares gibt der Mensch frei.

Beispiel: Der Agent sortiert Fotos in Alben, Löschen fragt nach.

Messwert: Wie oft greift der Mensch ein?

Run: in Grenzen allein

KI handelt in klaren Grenzen ohne Rückfrage, mit Monitoring und Notschalter.

Erst, wenn die Walk-Werte über Monate stabil sind.

Eine Stufe weiter erst, wenn die Messwerte der aktuellen Stufe stabil sind. Mit dokumentiertem Rückweg.

Abschlusspräsentationen: so läuft es

Regeln

- 5 Teams, je 10 Minuten Präsentation und 5 Minuten Diskussion
- Finale Abgabe: Tag „final“ bis 07.12.2026

Jede Person muss das ganze Projekt erklären können, nicht nur den eigenen Teil.

Quellen und Materialbasis Block 7

- Semantic Versioning 2.0.0 (semver.org); Git-Dokumentation: git tag; keepachangelog.com
- Sommerville: Software Engineering (10. Aufl.), Kap. 24–25 (Qualitäts- und Konfigurationsmanagement)
- Humble & Farley: Continuous Delivery (Release, Rollout, Canary, Blue-Green)
- IEEE 828 (Konfigurationsmanagement)
- Boehm (1991): Software Risk Management; PMBOK Guide; ISO 31000
- Anthropic: Building effective agents; Google: People + AI Guidebook